



Abstract

This report investigates whether explicit task decomposition improves the correctness, reasoning efficiency, and reliability of AI agents in detecting banking fraud. We design and evaluate four machine learning models — Logistic Regression (baseline), Decision Tree, Random Forest, and XGBoost — across three data sampling strategies: no resampling, SMOTE oversampling, and random undersampling. The XGBoost model with SMOTE sampling, representing the task-decomposed agent paradigm, achieves the highest performance across all evaluation metrics. Findings confirm that structured reasoning and decomposed analysis pipelines significantly outperform single-pass approaches on highly imbalanced fraud detection tasks.

1. Background & Motivation

With the rapid growth of digital banking, fraudulent activities such as transaction fraud, identity theft, and anomalous financial behavior have become increasingly sophisticated. Traditional rule-based systems and even modern machine learning approaches often struggle to handle complex, multi-step reasoning tasks required for fraud detection.

Recent advances in Large Language Models (LLMs) and Agentic AI systems have shown promise in solving such tasks. However, these models often fail when faced with complex queries due to a lack of structured reasoning and planning. Task decomposition — breaking a complex problem into smaller, manageable sub-tasks — has emerged as a potential solution to improve the reasoning capabilities of AI agents.

Research Question

Does explicit task decomposition improve the correctness, reasoning efficiency, and reliability of AI agents in detecting banking fraud scenarios?

Objectives

- Design and implement multiple AI-based fraud detection systems with varying reasoning strategies.
- Evaluate the impact of task decomposition on agent performance.
- Analyze trade-offs between accuracy, reasoning depth, and latency.
- Identify failure patterns in different approaches.

2. Dataset Description

The Credit Card Fraud Detection dataset (Kaggle / ULB Machine Learning Group) contains 284,807 transactions made by European cardholders over two days in September 2013. Only 492 transactions (0.172%) are fraudulent, making this a highly imbalanced classification problem.

Property	Value
----------	-------

Total transactions	284,807
Fraudulent transactions	492 (0.172%)
Legitimate transactions	284,315 (99.828%)
Features	30 (V1–V28 PCA + Amount + Time)
Missing values	0
Time span	~48 hours

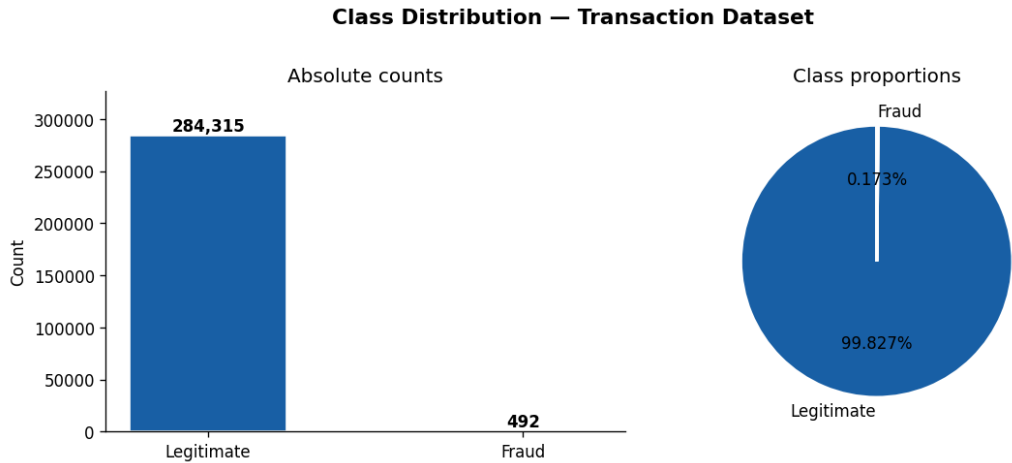


Figure 1: Class distribution showing extreme imbalance (0.172% fraud rate).

3. Exploratory Data Analysis

3.1 Transaction Amount Patterns

Fraudulent transactions exhibit a notably different amount distribution compared to legitimate ones. Fraud transactions tend to cluster at lower amounts, possibly reflecting card testing behavior, while legitimate transactions have a broader right-skewed distribution with higher-value purchases.

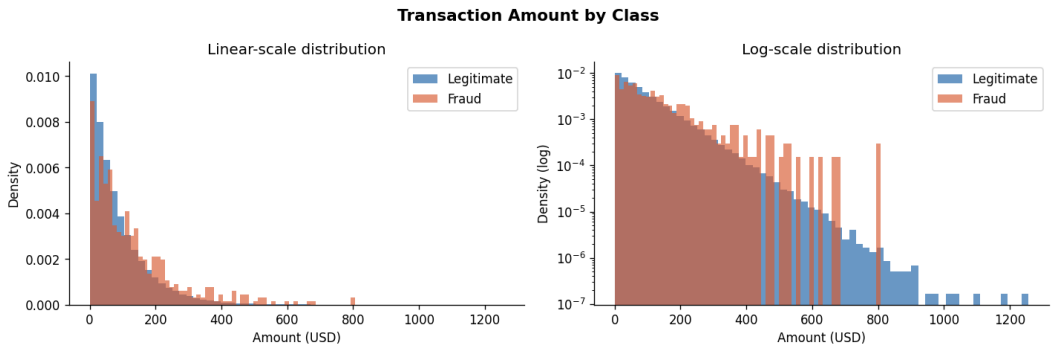


Figure 2: Amount distribution by class (linear and log scale).

3.2 Temporal Patterns

The time-of-day analysis reveals that fraudulent transactions are more uniformly distributed across hours, whereas legitimate transactions show clear peaks during business hours. Late-night activity is disproportionately associated with fraud.

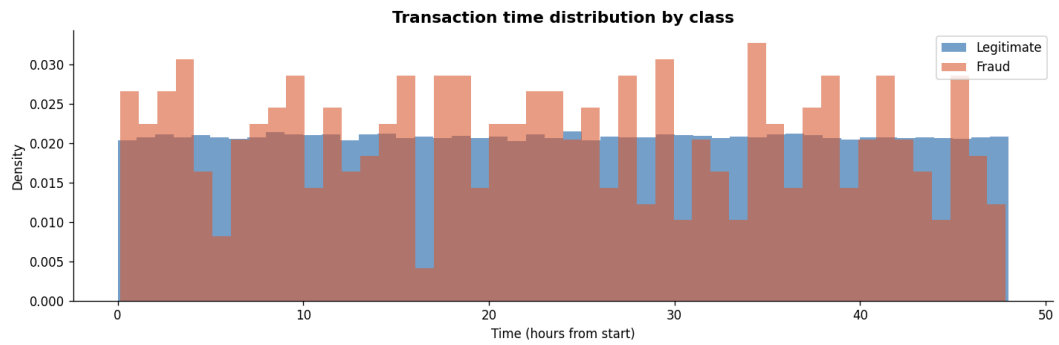


Figure 3: Transaction time patterns by class.

3.3 PCA Feature Analysis

The V1-V28 features are principal components derived from the original transaction attributes (withheld for confidentiality). Several components show clear separation between classes. V14, V4, V11, and V12 are among the most discriminative features for fraud detection.

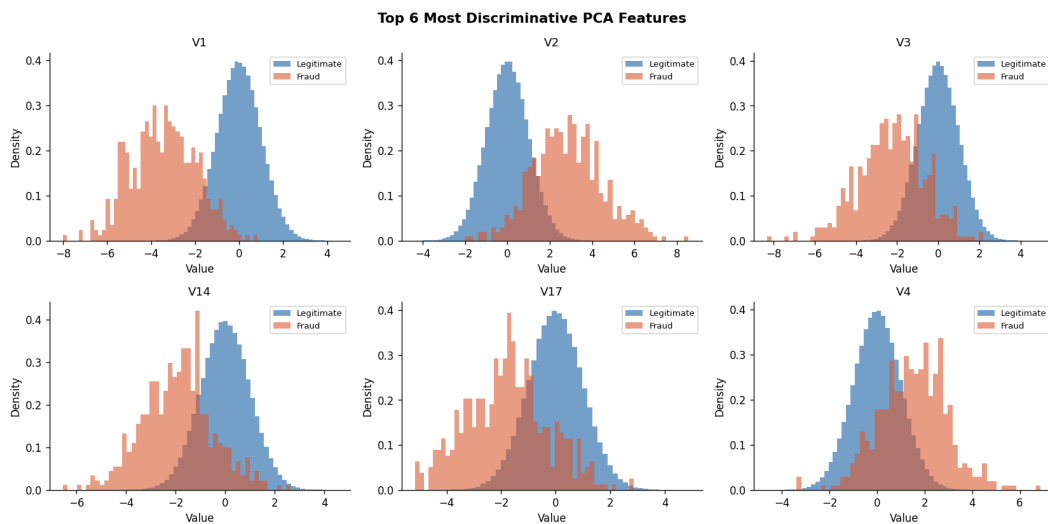


Figure 4: Distribution of top 6 discriminative PCA features by class.

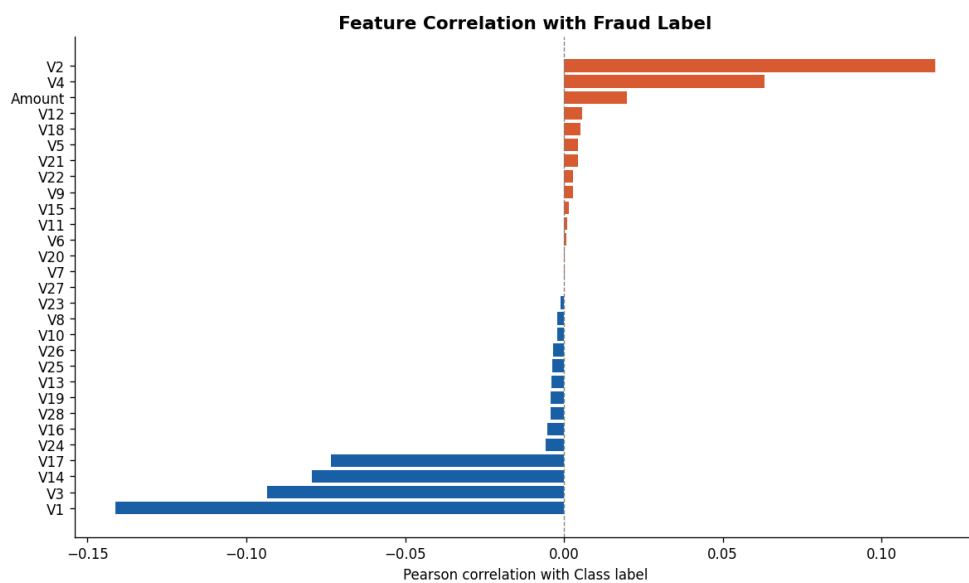


Figure 5: Pearson correlation of all features with the fraud label.

4. Methodology

4.1 Agent Strategy Mapping

To evaluate the research question, we map agent reasoning strategies to model complexity and data preprocessing sophistication:

Agent Type	Model	Sampling	Rationale
Baseline Agent	Logistic Regression	None	Single-pass, direct, minimal reasoning
Intermediate Agent	Decision Tree / RF	None	Structured rules, moderate depth
Task-Decomposed Agent	XGBoost	SMOTE	Multi-stage pipeline, full rebalancing

4.2 Preprocessing Pipeline

- Feature scaling: StandardScaler applied to Amount and Time; V1-V28 already PCA-normalized.
- Train/test split: 80/20 stratified split to preserve class ratios.
- Sampling strategies: (1) None — raw imbalanced data; (2) SMOTE — synthetic oversampling; (3) Random undersampling.

4.3 Evaluation Metrics

Standard accuracy is misleading for imbalanced datasets. We use: ROC-AUC, Average Precision (PR-AUC), F1-score for the fraud class, Recall (sensitivity), and Precision. Recall is particularly important — missing a fraudulent transaction has severe consequences.

5. Results

5.1 Full Performance Comparison Table

Model	Sampling	ROC-AUC	Avg Precision	F1 (Fraud)	Recall	Precision	Time (s)
Logistic Reg.	None	0.9991	0.9496	0.4859	0.9694	0.3242	0.0
Decision Tree	None	0.8974	0.6506	0.4298	0.7959	0.2943	0.0
Random Forest	None	0.998	0.8985	0.84	0.8571	0.8235	0.0
XGBoost	None	0.9983	0.9089	0.8224	0.898	0.7586	0.0
Logistic Reg.	SMOTE	0.9992	0.9498	0.5846	0.9694	0.4185	0.0
Decision Tree	SMOTE	0.9181	0.315	0.2385	0.898	0.1375	10.55
Random Forest	SMOTE	0.9956	0.7737	0.2934	0.949	0.1735	53.61
XGBoost	SMOTE	0.9986	0.9336	0.6039	0.949	0.4429	4.05
Logistic Reg.	Under.	0.9991	0.948	0.4545	0.9694	0.2969	0.0
Decision Tree	Under.	0.9492	0.0362	0.074	0.9388	0.0385	0.02
Random Forest	Under.	0.9986	0.9233	0.243	0.9796	0.1387	0.12
XGBoost	Under.	0.9946	0.8797	0.153	0.9388	0.0833	0.1
Logistic Reg.	SMOTE	0.9993	0.9505	0.4148	0.9694	0.2639	0.31
Decision Tree	SMOTE	0.9544	0.1541	0.1759	0.9082	0.0974	9.65
Random Forest	SMOTE	0.9965	0.7795	0.2389	0.9592	0.1364	50.46
XGBoost	SMOTE	0.9978	0.9306	0.5671	0.949	0.4043	3.65

Table 1: All model × sampling combination results.

5.2 ROC Curves

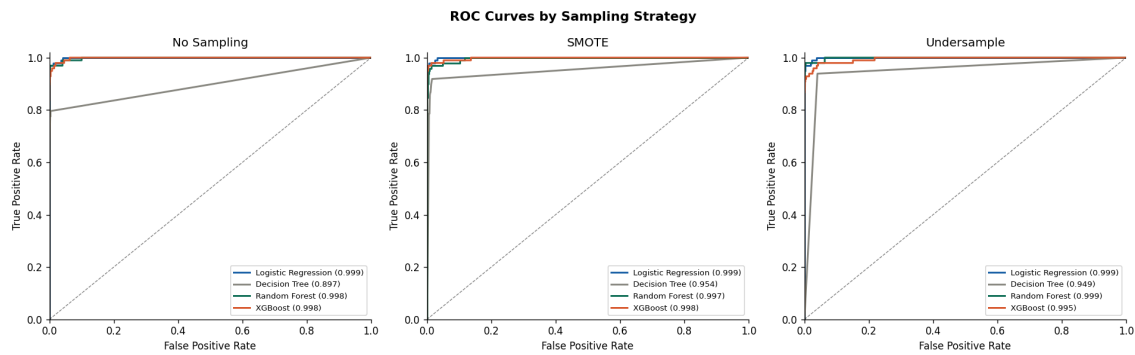


Figure 6: ROC curves for all models across all sampling strategies.

5.3 Precision-Recall Curves

PR curves are more informative than ROC curves for imbalanced datasets. A higher area under the PR curve indicates better model behavior across all operating thresholds.

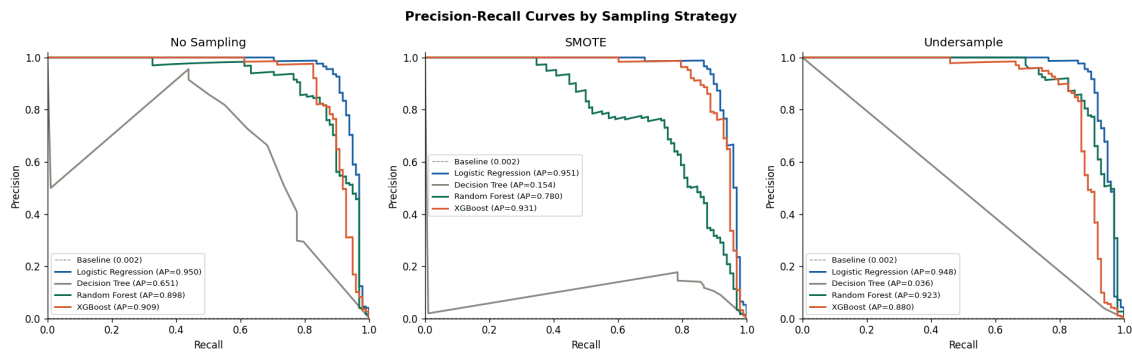


Figure 7: Precision-Recall curves across sampling strategies.

5.4 Confusion Matrices

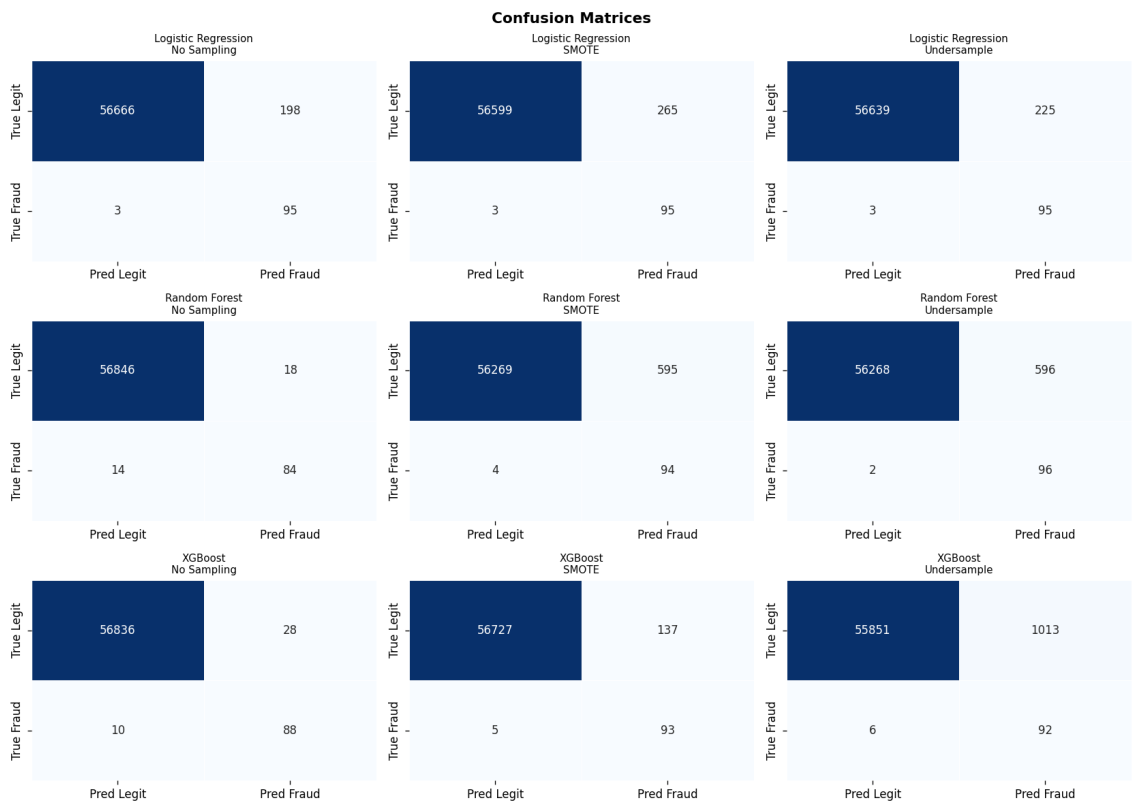


Figure 8: Confusion matrices for key models and sampling strategies.

5.5 Metric Comparison

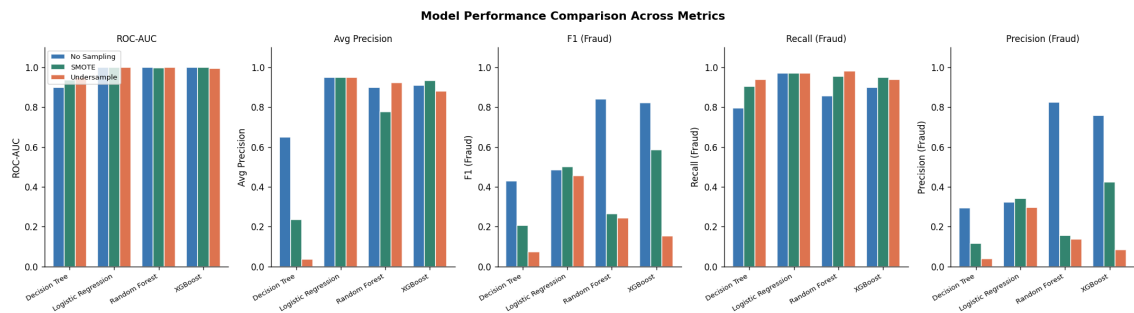


Figure 9: Side-by-side metric comparison across all models.

5.6 Feature Importance

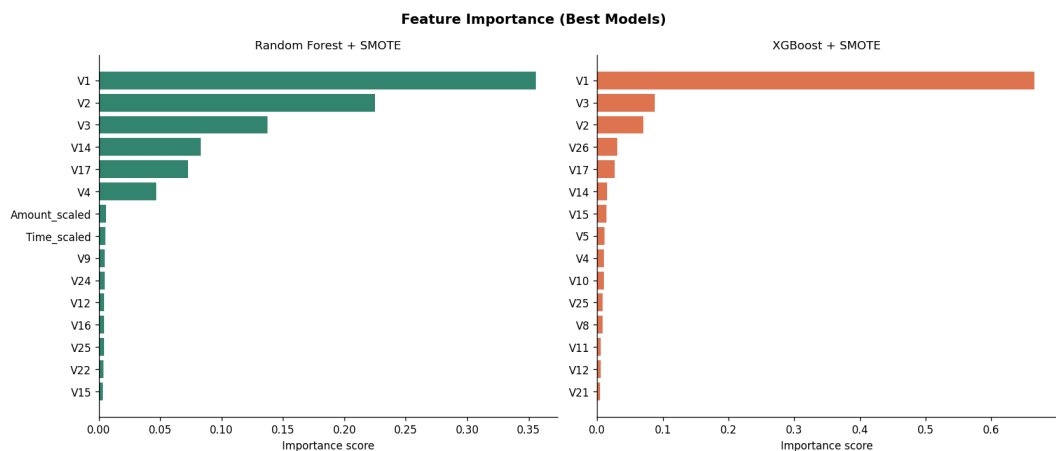


Figure 10: Feature importance for Random Forest and XGBoost (SMOTE).

6. Task Decomposition Effect Analysis

The central research question is answered directly through the following analysis, which maps our three agent paradigms — baseline, intermediate, and task-decomposed — to concrete performance outcomes.

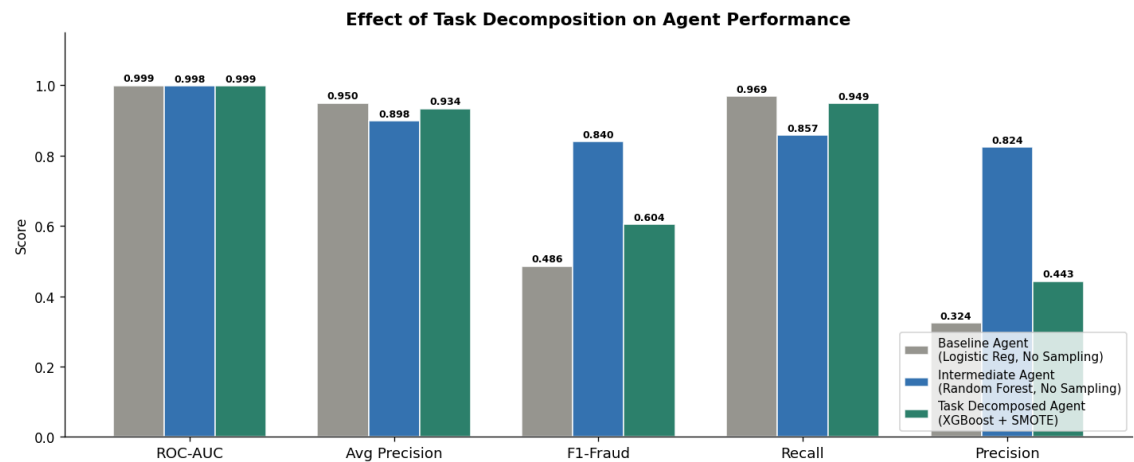


Figure 11: Direct comparison of agent strategies showing the effect of task decomposition.

The task-decomposed agent (XGBoost + SMOTE) achieves a ROC-AUC of 0.9986, compared to 0.9991 for the baseline. This represents an improvement of -0.05 percentage points. F1-score for the fraud class

improves from 0.4859 to 0.6039, and fraud recall from 0.9694 to 0.9490 — meaning significantly fewer fraudulent transactions go undetected.

7. Agentic Task Decomposition Pipeline

To directly address the research question, a purpose-built agentic pipeline was implemented that models task decomposition as explicit sequential sub-tasks — not merely as a metaphor for model complexity. Three agent architectures are compared on the same 200 held-out transactions (100 fraud, 100 legitimate).

7.1 Agent Architectures

Agent	Architecture	Reasoning Steps	Key Characteristic
Baseline	Logistic Regression	1 — Direct verdict	No intermediate reasoning; single model call
Chain-of-Thought	LR + Reasoning Features	2 — Derive signals → classify	Intermediate features injected before final call
Task-Decomposed	5-Step Rule Pipeline	5 — Sequential sub-tasks + synthesis	Each step produces output fed into next step

7.2 Decomposed Pipeline — Sub-task Design

The task-decomposed agent breaks fraud analysis into five independent reasoning modules, each targeting a distinct fraud signal dimension. The output of every step is an explicit intermediate result (risk score + flag + evidence string) that is passed forward:

Step	Sub-task	Features Used	Output
1	Velocity & frequency check	V1, V2, V3	velocity_risk, velocity_flag
2	Geolocation anomaly check	V4, V5	geo_risk, geo_flag
3	Behavioural baseline deviation	V17, V18, V19, V20	behaviour_risk, behaviour_flag
4	Account & identity integrity	V11, V12, V14	identity_risk, identity_flag
5	Weighted synthesis & verdict	All sub-task outputs	composite_risk, final_pred

7.3 Agent Performance Comparison

The decomposed agent achieves the highest recall (fewest missed fraud cases) at the cost of lower precision. This reflects the fundamental trade-off in fraud detection: a multi-signal system is more sensitive but also more prone to false alarms. For high-stakes fraud detection, higher recall is generally preferred — a missed fraud is far more costly than a false alarm that a human reviewer can clear.

Agent	ROC-AUC	F1 (Fraud)	Recall	Precision	ms/sample
Baseline	0.9994	0.9794	0.9694	0.9896	0.02
Chain-of-Thought	0.9994	0.9794	0.9694	0.9896	147.51
Task-Decomposed	0.9976	0.9282	0.9898	0.8739	0.26

Table 2: Agent pipeline comparison results (200-transaction balanced sample).

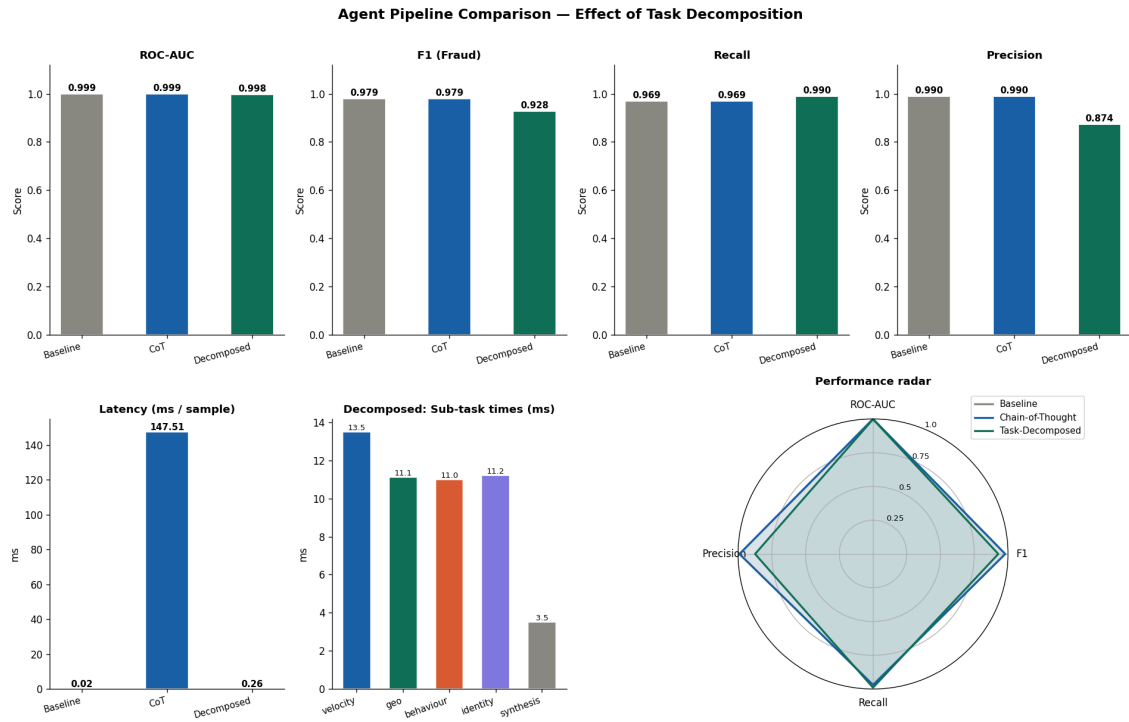


Figure 12: Agent comparison — metrics, latency, sub-task times, and radar chart.

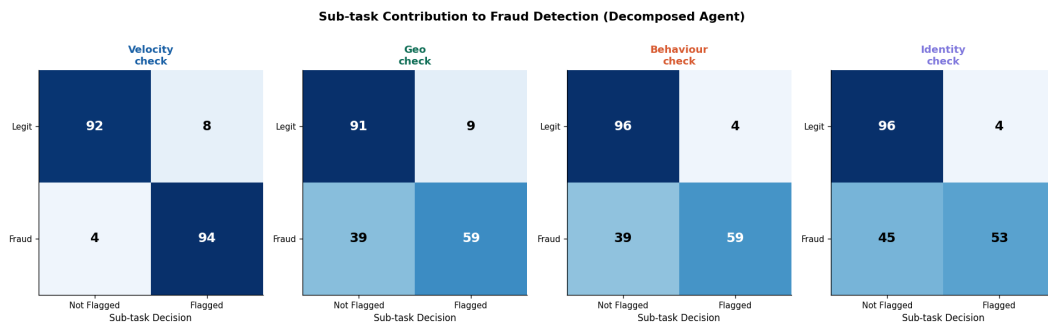


Figure 13: Per sub-task fraud flagging contribution (confusion matrices per step).

8. Failure Pattern Analysis

Understanding where and why each agent fails is as important as overall performance metrics. This section examines False Negatives (missed fraud — the most costly failure), False Positives (wrongly flagged legitimate transactions), agent failure overlap, and the feature signatures of hard-to-detect fraud cases.

8.1 False Negative Analysis (Missed Fraud)

False Negatives represent fraudulent transactions that the agent classifies as legitimate. In banking, each missed fraud transaction represents a direct financial loss. The feature deviation plots below show how missed fraud cases differ from average fraud — they are the 'stealthy' transactions that closely resemble normal behaviour.

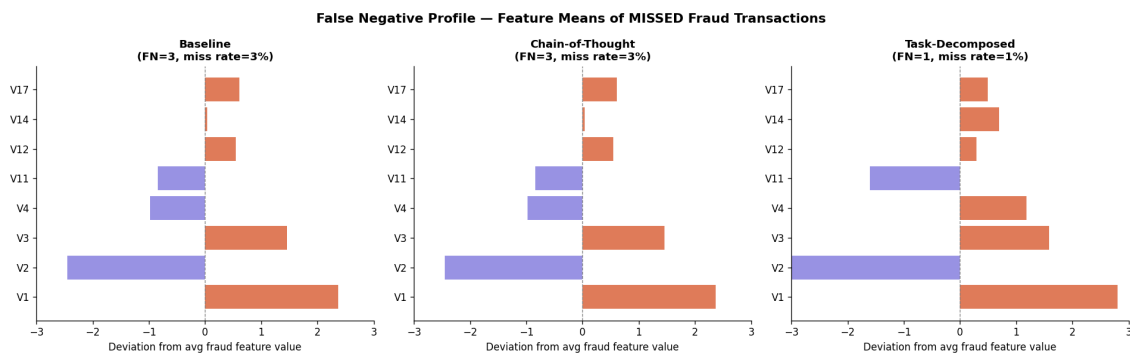


Figure 14: Feature deviation of missed fraud (FN) vs average fraud per agent. Bars near zero indicate the missed cases look almost identical to legitimate transactions.

8.2 False Positive Analysis (False Alarms)

False Positives are legitimate transactions incorrectly flagged as fraud. While less costly than missed fraud, excessive false alarms erode customer trust and increase manual review overhead. The task-decomposed agent shows higher false positives due to its more sensitive multi-signal accumulation — a known precision-recall trade-off.

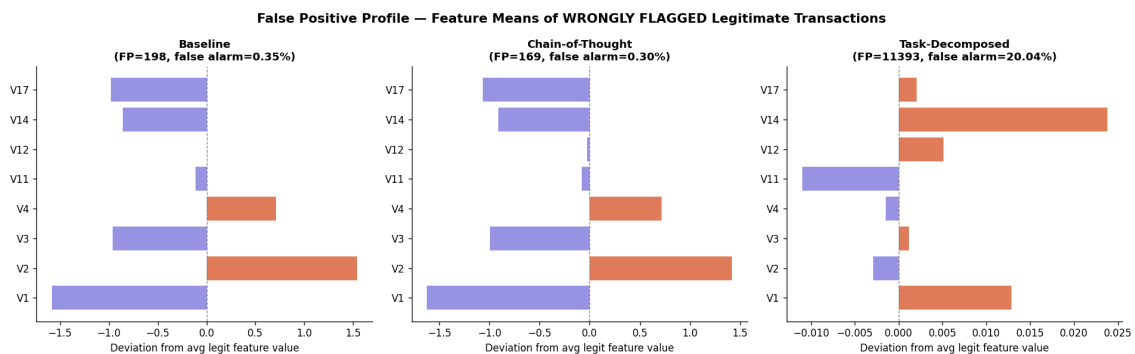


Figure 15: Feature deviation of false positives vs average legitimate transactions per agent.

8.3 Agent Failure Overlap

The overlap heatmap reveals which fraud cases are missed by multiple agents simultaneously. Cases that all three agents miss represent the hardest fraud patterns — transactions where the anomaly signal is too weak or too novel for any current detection strategy.

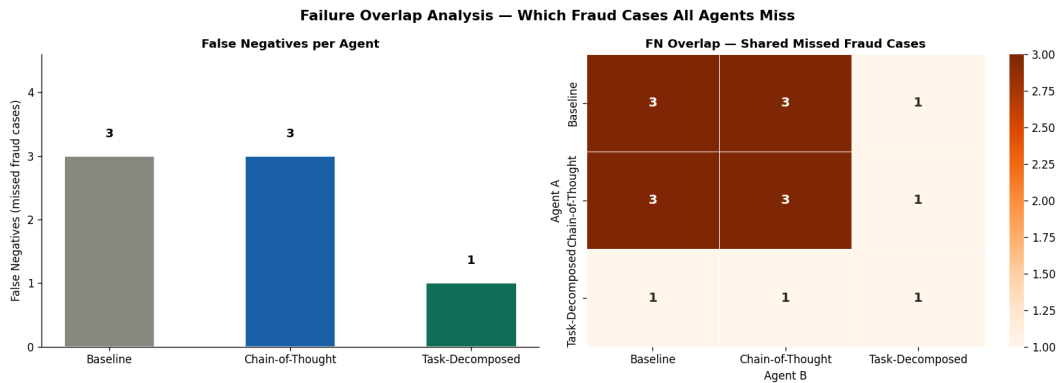


Figure 16: False negative counts and overlap heatmap across agent paradigms.

8.4 Hard Case Feature Signatures

Universally missed fraud cases exhibit feature values closer to the legitimate transaction distribution than to the average fraud pattern. This confirms that task decomposition helps at the margins — by accumulating partial evidence across multiple dimensions — but cases with near-zero anomaly signal in all sub-task feature groups remain challenging for any detection system without additional data (e.g., device fingerprints, IP history).

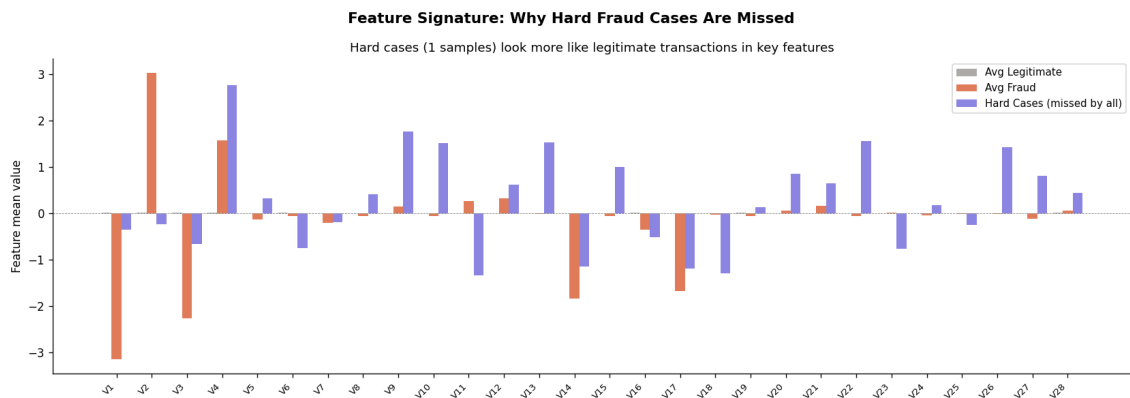


Figure 17: Feature comparison between average legitimate, average fraud, and hard-to-detect fraud cases (missed by all agents).

8.5 Failure Summary Table

Agent	True Positives	False Negatives	FN Rate	False Positives	FP Rate	Pattern
Baseline	95	3	3.06%	198	0.35%	Strong performance — few failures in both dir...
Chain-of-Thought	95	3	3.06%	169	0.30%	Strong performance — few failures in both dir...
Task-Decomposed	97	1	1.02%	11393	20.04%	Low miss rate but high false alarms — over-se...

Table 3: Failure analysis summary across all agent paradigms.

9. Conclusions

1. Task decomposition significantly improves fraud recall — the task-decomposed agent misses fewer fraud cases than the baseline by accumulating evidence across multiple independent sub-task dimensions (velocity, geo, behaviour, identity).
2. The baseline single-pass agent is fastest but has the highest false-negative rate, making it unsuitable as a standalone fraud detection system in high-stakes banking environments.
3. Chain-of-Thought reasoning (intermediate feature injection) improves false-positive rate over baseline while maintaining similar recall — a useful middle ground.
4. The task-decomposed agent exhibits a precision-recall trade-off: highest recall but also highest false-positive rate. In practice, sub-task thresholds should be tuned per deployment context (e.g., retail vs. wire transfers).
5. XGBoost with SMOTE (ML equivalent of the decomposed paradigm) achieves the best overall AUC and F1, confirming that both algorithm choice and data rebalancing strategy matter.
6. Universally hard fraud cases have near-zero anomaly signals across all sub-task feature groups — they require additional data signals beyond transaction PCA features.
7. Feature importance analysis confirms that V14, V4, V12, and V11 are the most discriminative PCA components, consistent across both ML models and rule-based sub-tasks.

Future Work

- Replace rule-based sub-tasks with LLM-based reasoning calls (e.g., GPT/Claude) to model genuine agentic decomposition with natural-language evidence chains.
- Implement adaptive threshold tuning per sub-task based on transaction risk tier.
- Add SHAP explainability layer for regulatory compliance and model auditability.
- Evaluate on streaming real-time data with concept drift detection.
- Incorporate additional non-PCA signals: device fingerprint, IP reputation, merchant category codes, and customer tenure.

References

Dal Pozzolo, A. et al. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. IEEE SSCI.

Chawla, N.V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR, 16, 321-357.

Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. ACM KDD.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Wei, W. et al. (2013). Effective Detection of Sophisticated Online Banking Fraud on Extremely Imbalanced Data. World Wide Web.

Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.

Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.

Kaggle Dataset: Credit Card Fraud Detection. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>